FOL:

```
def dls(graph, node, goal, depth, visited):
    if depth == 0 and node == goal:
        return [node]

    if depth > 0:
        visited.add(node)

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                path = dls(graph, neighbour, goal, depth - 1, visited)
                if path:
                    return [node] + path

    return None



def ids(graph, start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()
        path = dls(graph, start, goal, depth, visited)

        if path:
            print("Path Found:", path)
            print("Depth:", depth)
            return

    print("Goal not found within depth limit")
```

```
# -------- MAIN PROGRAM --------
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

start = 'A'
goal = 'F'
max_depth = 3

ids(graph, start, goal, max_depth)
```

A*

```
import heapq

def astar(graph, heuristic, start, goal):
    pq = [(heuristic[start], 0, start, [start])]  # (f, g, node, path)
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)
```

```
        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Optimal Path:", path)
            print("Total Cost:", g)
            return

        for neighbour, cost in graph.get(node, []):
            if neighbour not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbour]
                heapq.heappush(pq, (new_f, new_g, neighbour, path + [neighbour]))

    print("Goal not reachable")



# -------- MAIN PROGRAM --------
n = int(input("Enter number of vertices: "))
graph = {}

for i in range(n):
    node = input("Enter node: ")
    neighbours = int(input(f"Enter number of neighbours of {node}: "))
    graph[node] = []
```

```
    for j in range(neighbours):
        neighbour, cost = input("Enter neighbour and cost: ").split()
        graph[node].append((neighbour, int(cost)))

heuristic = {}
print("Enter heuristic values:")
for i in range(n):
    node = input("Node: ")
    h = int(input("Heuristic value: "))
    heuristic[node] = h

start = input("Enter start node: ")
goal = input("Enter goal node: ")

astar(graph, heuristic, start, goal)
```

Bayesian

```
p_A0 = float(input("Enter P(A=0): "))
p_A1 = 1 - p_A0
print("\nEnter P(B=0 | A=0) and P(B=0 | A=1):")
b00 = float(input("P(B=0 | A=0): "))
b01 = float(input("P(B=0 | A=1): "))
b10 = 1 - b00
b11 = 1 - b01
print("\nEnter P(C=0 | B=0) and P(C=0 | B=1):")
c00 = float(input("P(C=0 | B=0): "))
c01 = float(input("P(C=0 | B=1): "))
c10 = 1 - c00
```

```
c11 = 1 - c01
P_A = [p_A0, p_A1]
P_B_given_A = [[b00, b01], [b10, b11]]
P_C_given_B = [[c00, c01], [c10, c11]]

def joint(a, b, c):
    return P_A[a] * P_B_given_A[b][a] * P_C_given_B[c][b]
print("\nJoint Probability Table:\n")
for a in [0, 1]:
    for b in [0, 1]:
        for c in [0, 1]:
            print(f"P(A={a}, B={b}, C={c}) = {joint(a,b,c):.4f}")
```

bfs

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    print("BFS Traversal:", end=" ")

    while queue:
        node = queue.popleft()
        print(node, end=" ")
```

```
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)



# -------- MAIN PROGRAM --------
n = int(input("Enter number of vertices: "))
graph = {}

for i in range(n):
    node = input("Enter node: ")
    neighbours = input(f"Enter neighbours of {node} (space separated): ").split()
    graph[node] = neighbours

start_node = input("Enter starting node: ")

bfs(graph, start_node)
```

blockworld

```
def print_state(state):
    print(state)
def is_goal(state, goal):
    return state == goal
def state_to_tuple(state):
    return tuple(tuple(stack) for stack in state)
def move_block(state, from_stack, to_stack):
```

```
    new_state = [stack[:] for stack in state]
    if not new_state[from_stack]:
        return None
    block = new_state[from_stack].pop()
    if to_stack == -1:
        new_state.append([block])
    else:
        new_state[to_stack].append(block)
    new_state = [s for s in new_state if s]
    return new_state
def block_world(initial, goal):
    visited = set()
    current = initial
    steps = 0
    print("\nInitial State:")
    print_state(current)
    while not is_goal(current, goal):
        visited.add(state_to_tuple(current))
        moved = False
        for i in range(len(current)):
            for j in range(len(current)):
                if i != j:
                    new_state = move_block(current, i, j)
                    if new_state and state_to_tuple(new_state) not in visited:
                        print(f"\nStep {steps+1}: Move block from stack {i} to {j}")
                        print_state(new_state)
                        current = new_state
                        steps += 1
```

```
                moved = True
                break
            if moved:
                break
        if not moved:
            for i in range(len(current)):
                new_state = move_block(current, i, -1)
                if new_state and state_to_tuple(new_state) not in visited:
                    print(f"\nStep {steps+1}: Move block from stack {i} to table")
                    print_state(new_state)
                    current = new_state
                    steps += 1
                    moved = True
                    break
        if not moved:
            print("\nNo solution found (stuck state).")
            return
    print("\nGoal State Reached!")
    print_state(current)

initial = eval(input("Enter initial state: "))
goal = eval(input("Enter goal state: "))

block_world(initial, goal)
```

dfs

```
def dfs(graph, start, visited):
    visited.add(start)
    print(start, end=" ")

    for neighbour in graph.get(start, []):  # safe access
        if neighbour not in visited:
            dfs(graph, neighbour, visited)



# -------- MAIN PROGRAM --------
n = int(input("Enter number of vertices: "))
graph = {}

for i in range(n):
    node = input("Enter node: ")
    neighbours = input(f"Enter neighbours of {node} (space separated): ").split()
    graph[node] = neighbours

start_node = input("Enter starting node: ")

print("DFS Traversal:", end=" ")
visited = set()
dfs(graph, start_node, visited)
```

greedy

```
import heapq
```

```
def greedy_best_first_search(graph, heuristic, start, goal):
    visited = set()
    pq = [(heuristic[start], start, [start])]  # (heuristic, node, path)

    while pq:
        h, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Path Found:", path)
            return

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                heapq.heappush(pq, (heuristic[neighbour], neighbour, path + [neighbour]))

    print("Goal not reachable")



# -------- MAIN PROGRAM --------
n = int(input("Enter number of vertices: "))
graph = {}
```

```
for i in range(n):
    node = input("Enter node: ")
    neighbours = input(f"Enter neighbours of {node} (space separated): ").split()
    graph[node] = neighbours

heuristic = {}
print("Enter heuristic values:")
for i in range(n):
    node = input("Node: ")
    h = int(input("Heuristic value: "))
    heuristic[node] = h

start = input("Enter start node: ")
goal = input("Enter goal node: ")

greedy_best_first_search(graph, heuristic, start, goal)
```

ids

```
def dls(graph, node, goal, depth, visited):
    if depth == 0 and node == goal:
        return [node]

    if depth > 0:
        visited.add(node)

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                path = dls(graph, neighbour, goal, depth - 1, visited)
```

```
            if path:
                return [node] + path

    return None


def ids(graph, start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()
        path = dls(graph, start, goal, depth, visited)

        if path:
            print("Path Found:", path)
            print("Depth:", depth)
            return

    print("Goal not found within depth limit")


# -------- MAIN PROGRAM --------
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
```

```
start = 'A'
goal = 'F'
max_depth = 3

ids(graph, start, goal, max_depth)
```

monkey

```
def monkey_banana():

    monkey = input("Enter monkey position: ")
    box = input("Enter box position: ")
    banana = input("Enter banana position: ")
    print("\nInitial State:")
    print("Monkey:", monkey, "| Box:", box, "| Banana:", banana)
    print("\nSteps:\n")
    if monkey != box:
        print("Step 1: Monkey moves from", monkey, "to", box)
        monkey = box
    if box != banana:
        print("Step 2: Monkey pushes box from", box, "to", banana)
        box = banana
    print("Step 3: Monkey climbs onto the box")


    print("Step 4: Monkey grabs the banana")
    print("\nGoal Achieved: Monkey got the banana!")
```

```
monkey_banana()
```

nqueens

```
def is_safe(board, row, col, n):

    for i in range(row):
        if board[i][col] == 1:
            return False



    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1
    return True

def solve_n_queens(board, row, n):

    if row == n:
```

```
        return True

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1

            if solve_n_queens(board, row + 1, n):
                return True

            board[row][col] = 0

    return False
def print_board(board, n):
    print("\nSolution Board:\n")
    for i in range(n):
        for j in range(n):
            if board[i][j] == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
if __name__ == "__main__":
    print("N-QUEENS PROBLEM USING BACKTRACKING\n")
    n = int(input("Enter number of queens (N): "))
    board = [[0 for _ in range(n)] for _ in range(n)]
    if solve_n_queens(board, 0, n):
        print_board(board, n)
    else:
```

```
    print("\nNo solution exists.")
```

tictac

```
import math

board = [" " for _ in range(9)]

def print_board():
    for i in range(0,9,3):
        print(board[i], "|", board[i+1], "|", board[i+2])
        if i < 6:
            print("--+---+--")

def check_winner():
    win_positions = [
        (0,1,2),(3,4,5),(6,7,8),
        (0,3,6),(1,4,7),(2,5,8),
        (0,4,8),(2,4,6)
    ]

    for a,b,c in win_positions:
        if board[a]==board[b]==board[c] and board[a]!=" ":
            return board[a]

    if " " not in board:
        return "Draw"
```

```
    return None

def minimax(is_max, alpha, beta):

    result = check_winner()

    if result == "X":
        return 1
    elif result == "O":
        return -1
    elif result == "Draw":
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if board[i]==" ":
                board[i]="X"
                val=minimax(False,alpha,beta)
                board[i]=" "
                best=max(best,val)
                alpha=max(alpha,val)
                if beta<=alpha:
                    break
        return best

    else:
        best = math.inf
```

```
        for i in range(9):
            if board[i]==" ":
                board[i]="O"
                val=minimax(True,alpha,beta)
                board[i]=" "
                best=min(best,val)
                beta=min(beta,val)
                if beta<=alpha:
                    break
        return best

def best_move():
    best_val=-math.inf
    move=-1

    for i in range(9):
        if board[i]==" ":
            board[i]="X"
            val=minimax(False,-math.inf,math.inf)
            board[i]=" "

            if val>best_val:
                best_val=val
                move=i

    return move

while True:
```

```
    print_board()
    pos=int(input("Enter position (0-8): "))

    if board[pos]==" ":
        board[pos]="O"

    if check_winner():
        break

    ai_move=best_move()
    board[ai_move]="X"

    if check_winner():
        break

print_board()
print("Winner:",check_winner())
```

tsp

```
import copy

def calculate_cost(tour, distance):
    cost = 0
    for i in range(len(tour) - 1):
        cost += distance[tour[i]][tour[i + 1]]

    cost += distance[tour[-1]][tour[0]]
```

```
    return cost

def generate_neighbors(tour):
    neighbors = []
    for i in range(1, len(tour) - 1):
        for j in range(i + 1, len(tour)):
            new_tour = copy.deepcopy(tour)
            new_tour[i], new_tour[j] = new_tour[j], new_tour[i]
            neighbors.append(new_tour)
    return neighbors

def hill_climbing(distance):
    n = len(distance)

    current_tour = list(range(n))
    current_cost = calculate_cost(current_tour, distance)

    print("\nInitial Tour:", current_tour)
    print("Initial Cost:", current_cost)
    print("-" * 50)

    while True:
        neighbors = generate_neighbors(current_tour)

        best_neighbor = current_tour
        best_neighbor_cost = current_cost

        for neighbor in neighbors:
```

```
            cost = calculate_cost(neighbor, distance)
            print("Checking Neighbor:", neighbor, "Cost:", cost)

            if cost < best_neighbor_cost:
                best_neighbor = neighbor
                best_neighbor_cost = cost

        print("-" * 50)

        if best_neighbor_cost >= current_cost:
            break

        current_tour = best_neighbor
        current_cost = best_neighbor_cost
        print("Move Accepted")
        print("New Tour:", current_tour)
        print("New Cost:", current_cost)
        print("-" * 50)
    return current_tour, current_cost
if __name__ == "__main__":
    n = int(input("Enter number of cities: "))

    print("Enter distance matrix row by row:")
    distance = []

    for i in range(n):
        row = list(map(int, input().split()))
        distance.append(row)
```

```
    final_tour, final_cost = hill_climbing(distance)

    print("\nFinal Solution")
    print("Best Tour:", final_tour)
    print("Best Cost:", final_cost)
```

ucs

```
import heapq

def ucs(graph, start, goal):
    pq = [(0, start, [])]   # (cost, node, path)
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        path = path + [node]

        if node == goal:
            print("Least Cost Path:", path)
            print("Total Cost:", cost)
            return
```

```
        for neighbour, weight in graph.get(node, []):
            if neighbour not in visited:
                heapq.heappush(pq, (cost + weight, neighbour, path))

    print("Goal not reachable")

# -------- MAIN PROGRAM --------
n = int(input("Enter number of vertices: "))
graph = {}

for i in range(n):
    node = input("Enter node: ")
    neighbours_count = int(input(f"Enter number of neighbours of {node}: "))
    graph[node] = []

    for j in range(neighbours_count):
        neighbour, cost = input("Enter neighbour and cost: ").split()
        graph[node].append((neighbour, int(cost)))

start = input("Enter start node: ")
goal = input("Enter goal node: ")

ucs(graph, start, goal)
```

waterjug

```
import heapq
import math
```

```
def is_solvable(capA, capB, target):
    """Check if solution is mathematically possible"""
    if target > max(capA, capB):
        return False
    return target % math.gcd(capA, capB) == 0

def water_jug_a_star(capA, capB, target):
    start = (0, 0)

    pq = []
    heapq.heappush(pq, (0, 0, start, []))

    visited = set()

    while pq:
        f, g, (a, b), path = heapq.heappop(pq)

        # Goal check
        if a == target or b == target:
            return path + [(a, b)]

        if (a, b) in visited:
            continue

        visited.add((a, b))

        next_states = [
```

```
            (capA, b), # Fill A
            (a, capB), # Fill B
            (0, b), # Empty A
            (a, 0), # Empty B
            # Pour A -> B
            (a - min(a, capB - b), b + min(a, capB - b)),
            # Pour B -> A
            (a + min(b, capA - a), b - min(b, capA - a))
        ]

        for na, nb in next_states:
            if (na, nb) not in visited:
                h = min(abs(na - target), abs(nb - target))
                heapq.heappush(
                pq,
                (g + 1 + h, g + 1, (na, nb), path + [(a, b)]))
    return None
if __name__ == "__main__":
    print("\nWATER JUG PROBLEM USING A* ALGORITHM\n")

    capA = int(input("Enter capacity of Jug A: "))
    capB = int(input("Enter capacity of Jug B: "))
    target = int(input("Enter target amount: "))

    if not is_solvable(capA, capB, target):
        print("\nNo solution possible mathematically.")
    else:
        solution = water_jug_a_star(capA, capB, target)
```

```
    if solution:
        print("\nSolution Steps:")
        step = 0
        for a, b in solution:
            print(f"Step {step}: Jug A = {a}, Jug B = {b}")
            step += 1
    else:
        print("\nNo solution found.")
```

Wumpus

```
size = 4

world = [["Safe" for _ in range(size)] for _ in range(size)]

pits = [(1,2),(2,2)]
wumpus = (3,1)

for p in pits:
    world[p[0]][p[1]] = "Pit"

world[wumpus[0]][wumpus[1]] = "Wumpus"
def neighbors(x,y):
    moves=[(-1,0),(1,0),(0,-1),(0,1)]
    cells=[]
    for dx,dy in moves:
        nx,ny=x+dx,y+dy
```

```
        if 0<=nx<size and 0<=ny<size:
            cells.append((nx,ny))
    return cells

def percept():
    breeze=set()
    stench=set()

    for x,y in pits:
        for n in neighbors(x,y):
            breeze.add(n)

    for n in neighbors(wumpus[0],wumpus[1]):
        stench.add(n)

    return breeze,stench

breeze, stench = percept()

print("Breeze Cells:", breeze)
print("Stench Cells:", stench)

print("\nWorld State")

for i in range(size):
    for j in range(size):
        print(world[i][j], end="\t")
    print()
```